

AhamX Bodhi

Where I Becomes Infinite

Level 4: Integration

Module 2: Generative AI – Core Concepts & Architecture
Prof. Sashikumaar Ganesan · IISc Bengaluru



Session Overview

Session Details

- **Module:** 2 – Core Concepts & Architecture
- **Duration:** 10 minutes
- **Bloom's Level:** B3 (Apply)
- **Difficulty:** L2 (Intermediate)
- **Sequence:** Concept 4 of Module 2

The Central Question

A working AI application is not a production system. Prototypes break under load, drift silently, and haemorrhage money at scale.

What separates a prototype builder from a production engineer?

This session answers that with the disciplines of LLMOps and production integration.



Learning Objectives

By the End of This Session You Will Be Able To

1. **Define** Level 4 Integration and explain what distinguishes it from Level 3 Applications
2. **Describe** the LLMOps lifecycle: build, deploy, monitor, iterate
3. **Identify** the core components of a production AI stack: serving, containerization, observability
4. **Explain** inference optimisation techniques – quantisation, batching, caching – and their trade-offs
5. **Recognise** safety guardrails, cost controls, and CI/CD patterns in deployed production systems



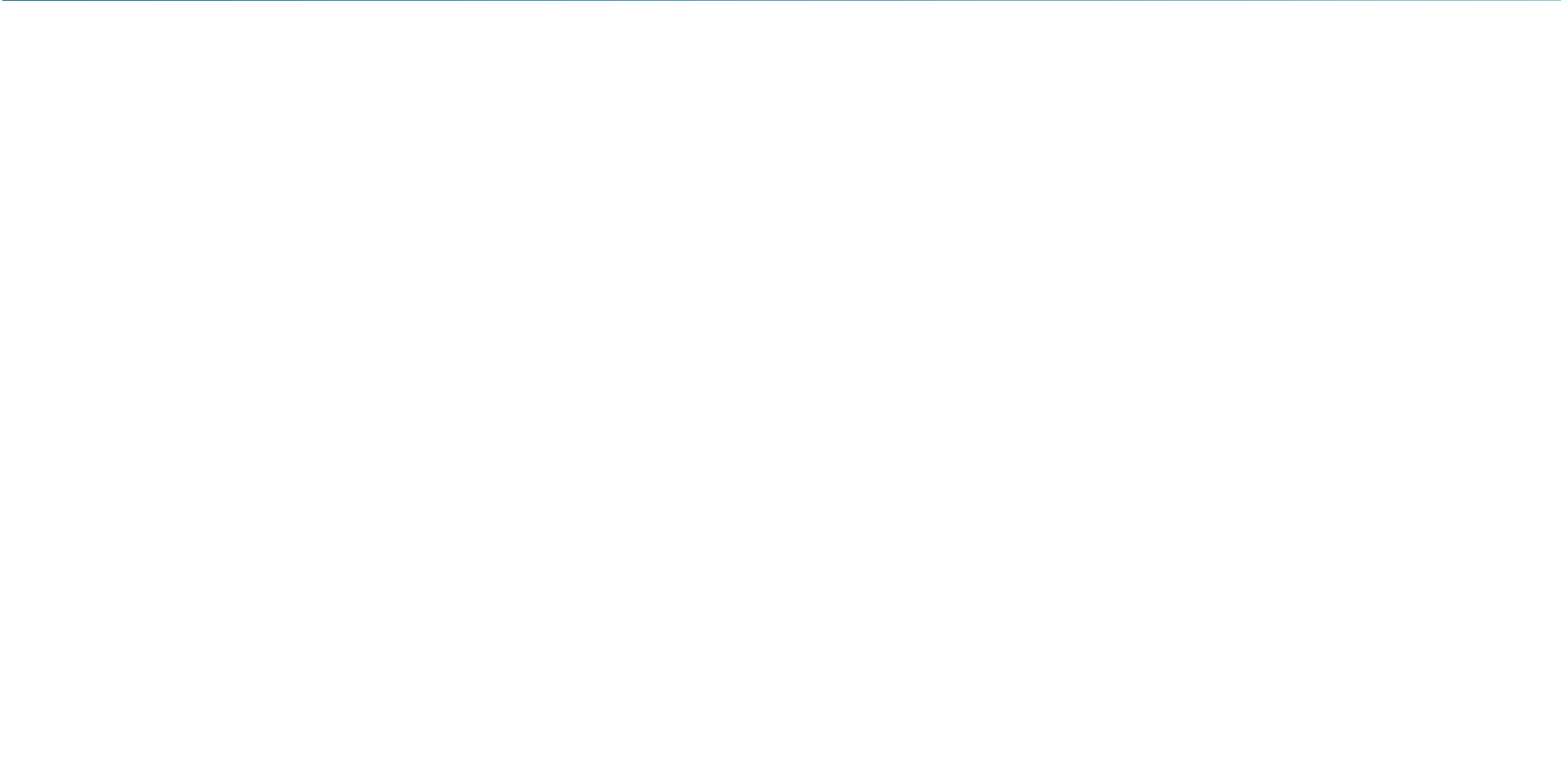
Prerequisites and Context

You Should Already Know

- Foundation models and their properties
- The three adaptation strategies (Level 2)
- AI agent architecture and ReAct (Level 3)
- The Five-Level GenAI Framework overview
- Basic software engineering concepts (APIs, containers)

This Session Unlocks

- LLMOps Fundamentals (Module 9)
- Docker and Containerization for AI (Module 9)
- Inference Optimisation in Depth (Module 9)
- Safety Guardrails and Evaluation (Module 9)
- Capstone: Production System Design (Module 10)





What Is Level 4 Integration?

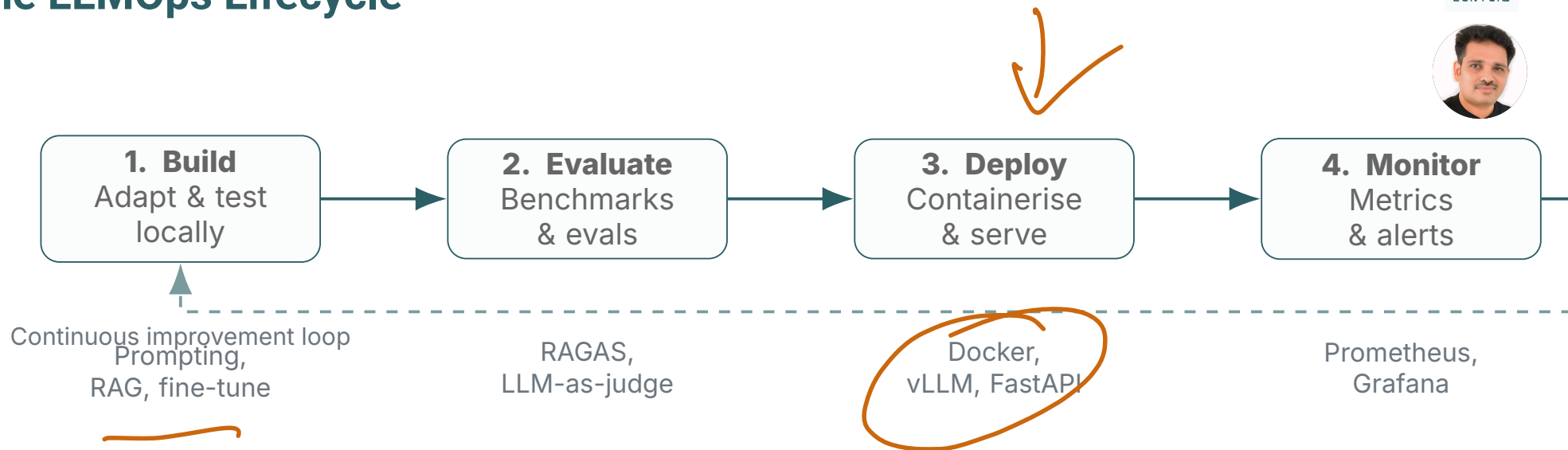
Definition

Level 4 Integration is the discipline of embedding AI systems reliably, securely, and cost-efficiently into existing products, workflows, and business processes – and keeping them working at scale over time. At this level, AI is no longer a standalone demo: it is a **production component** with SLAs, cost budgets, monitoring dashboards, and upgrade pipelines. Users may not even realise they are interacting with AI.

The Four Engineering Shifts from L3 to L4

- **Correctness** → **Reliability**: From “it works once” to “it works 99.9% of the time”
- **Speed** → **Throughput**: From single-user demo to thousands of concurrent requests
- **Ad-hoc cost** → **Cost per query**: From “it runs” to “each call costs \$0.002”
- **Static** → **Evolving**: From frozen code to continuous evaluation, versioning, and redeployment

The LLMOps Lifecycle



LLMOps is the practice of managing this cycle reliably. Unlike classical MLOps, it must handle **open-ended outputs**, **prompt-as-code**, and **unpredictable generation costs**.





Containerisation: Docker for AI Systems

Why Containers for AI?

A model that runs on a developer's laptop with specific CUDA drivers, Python version, and library dependencies will often fail on a different machine or cloud instance. **Docker containers** bundle the application, its runtime environment, all dependencies, and the model artefacts into a single portable image that runs identically everywhere – from a laptop to a cloud VM to a Kubernetes cluster.

A Production AI Docker Image Contains

- **Base image:** CUDA-enabled Ubuntu (e.g. `nvidia/cuda:12.2-runtime`)
- **Runtime:** Python 3.11, PyTorch, Transformers, vLLM or TGI serving framework
- **Model artefacts:** Quantised checkpoint or pointer to HuggingFace Hub
- **API layer:** FastAPI application exposing `/generate` and `/health` endpoints
- **Config:** Environment variables for model path, max tokens, GPU memory fraction

LLM Serving Frameworks



Open-Source Serving

- 
- **vLLM**: PagedAttention for high-throughput; continuous batching; supports Llama, Mistral, Gemma
 - **TGI (Text Generation Inference)**: HuggingFace's production server; gRPC and HTTP
 - **Ollama**: Local single-node serving; quantised GGUF models; developer-first
 - **LiteLLM**: Universal proxy – one API for 100+ LLMs from any provider

Cloud Managed Serving

- **AWS SageMaker**: Managed endpoint with auto-scaling and A/B testing
- **Azure ML**: Online endpoints with blue/green deployments
- **Google Vertex AI**: Gemini and open-weight model hosting with monitoring
- **Together.ai / Fireworks**: Inference-optimised cloud for open-weight models
- **Replicate**: Serverless model hosting; pay per prediction





Inference Optimisation: Quantisation

What Is Quantisation?

Quantisation reduces the numerical precision of model weights from 32-bit floating point (FP32) to lower-precision formats – typically INT8 or INT4 – shrinking memory footprint and accelerating computation with minimal quality degradation. A Llama 3 8B model in FP16 requires ~16 GB of VRAM; in INT4 it fits in ~4.5 GB – runnable on a consumer GPU.

Quantisation Methods in Practice

- **GPTQ:** Post-training weight quantisation using calibration data; minimal accuracy loss at INT4
- **AWQ (Activation-Aware Weight Quantisation):** Identifies and preserves the most important weights
- **BitsAndBytes:** HuggingFace library enabling 4-bit and 8-bit loading with `load_in_4bit=True`
- **GGUF (llama.cpp):** CPU-optimised quantised format; runs Llama 3 on a MacBook without a GPU



Batching and KV Cache Optimisation

Continuous Batching

- Naive serving processes one request at a time; GPU is mostly idle
- **Continuous batching** (vLLM) dynamically inserts new requests into the active batch as tokens complete
- Result: 5–20× throughput improvement over static batching
- Required for any production system with concurrent users

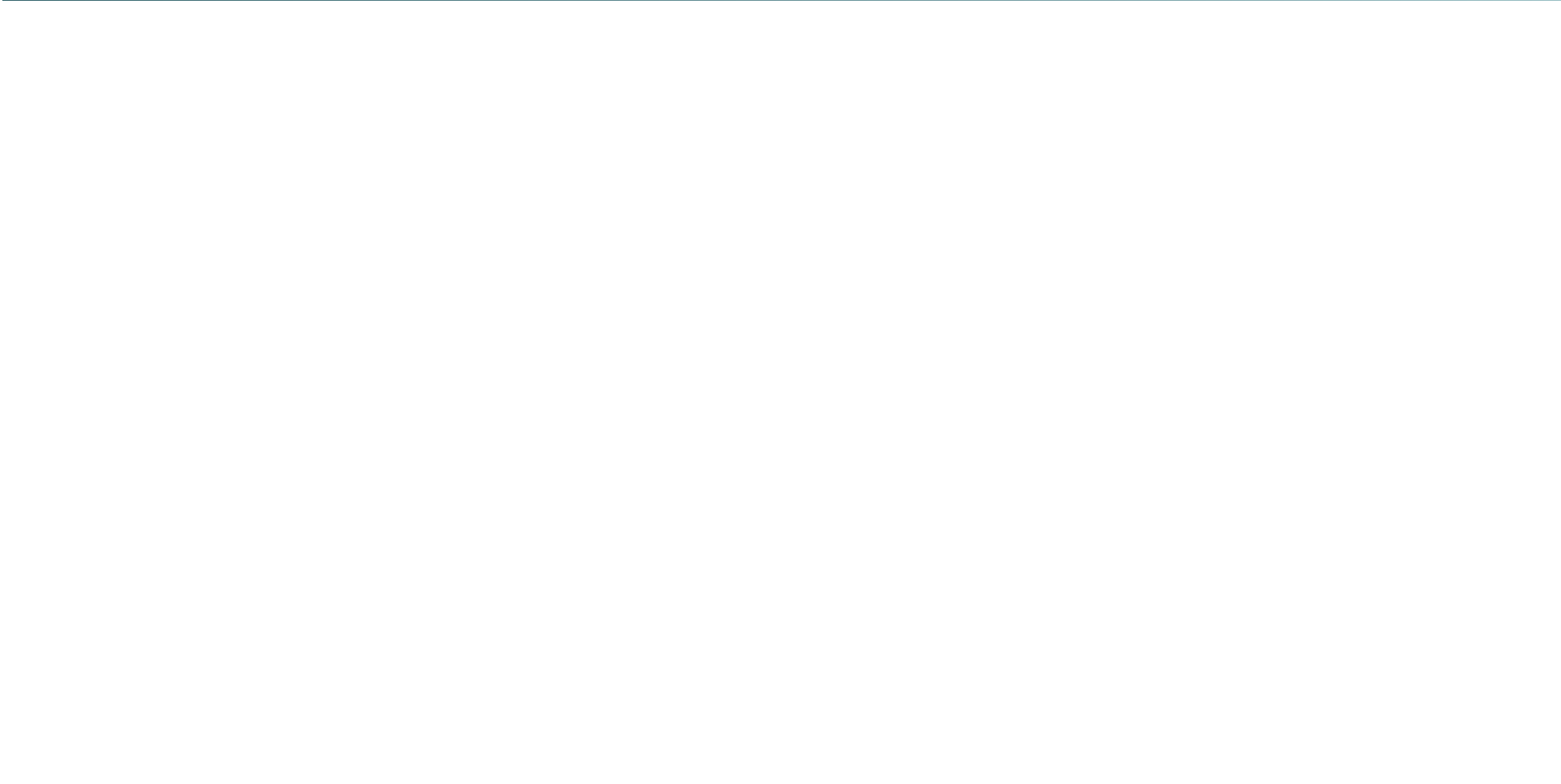
KV Cache and PagedAttention

- Self-attention computes key-value tensors for every token in context
- **KV cache** stores these tensors so they are not recomputed on each new token
- **PagedAttention** (vLLM): manages KV cache in non-contiguous memory pages, eliminating fragmentation
- Enables serving long-context models (128K tokens) without running out of VRAM



Inference Optimisation: Technique Comparison

Technique	What It Reduces	Quality Impact	Best For
INT4 Quantisation	VRAM by $\sim 4\times$	$<1\%$ on most benchmarks	Consumer GPU / edge deployment
Continuous Batching	Latency per user	None	Multi-user API endpoints
KV Cache / Page-Attention	GPU memory fragmentation	None	Long-context models
Speculative Decoding	Token generation time	None	Low-latency chat applications
Flash Attention 2	Attention memory footprint	None	Training and long-context inference
Model Distillation	Model size ($\sim 10\times$)	Moderate	Mobile / IoT / latency-critical





Monitoring and Observability for LLMs

Why LLM Monitoring Is Different

Classical software monitoring tracks CPU, memory, and error rates. LLM monitoring must additionally track **output quality** (hallucination rate, relevance), **prompt drift** (changes in user query distributions over time), **cost per query**, and **latency under varying context lengths**. A system can have zero HTTP errors while silently producing degraded AI outputs.

The Four Monitoring Layers

- **Infrastructure:** GPU utilisation, VRAM usage, pod health – Prometheus + Grafana
- **API:** Request rate, latency percentiles (p50/p95/p99), error codes – Prometheus + FastAPI middleware
- **Output quality:** Hallucination rate, faithfulness score, user feedback – RAGAS, LLM-as-judge
- **Cost:** Tokens consumed per request, cost per user, daily burn rate – LangSmith, Helicone, custom logging



Cost Control in Production AI

Where Costs Come From

- **Input tokens:** System prompt + user query + retrieved context
- **Output tokens:** Generated response (typically more expensive per token)
- **GPU compute:** On-prem or cloud GPU hours for self-hosted models
- **Embedding calls:** Every RAG query embeds the user input
- **Reranking:** Cross-encoder reranking of retrieved chunks

Cost Reduction Strategies

- **Prompt caching:** Cache repeated system prompt tokens (Anthropic: 90% cost reduction on cached prefix)
- **Model routing:** Route simple queries to GPT-4o-mini; complex ones to GPT-4o
- **Context compression:** Summarise conversation history before injection
- **Output length control:** Set `max_tokens`; use structured output to avoid verbose prose
- **Batching:** Process non-latency-critical requests in batch API (50% discount on OpenAI)

Safety Guardrails in Production



Why Guardrails Are Mandatory at Level 4

A model fine-tuned for helpfulness will generate responses to virtually any input. At production scale, this creates liability, brand risk, and regulatory exposure. **Safety guardrails** are software layers that intercept inputs and outputs, enforcing content policies, preventing prompt injection attacks, and ensuring compliance with regulations such as the EU AI Act and HIPAA – **independently of the model's own training**.

Guardrail Architecture Layers

- **Input screening:** Detect jailbreaks, prompt injection, PII leakage before the model sees the request
- **Topic restriction:** Block off-topic queries (e.g. a customer support bot refusing to discuss competitors)
- **Output filtering:** Scan generated text for harmful content, PII, hallucinated citations
- **Rate limiting:** Per-user and per-tenant quotas to prevent abuse and uncontrolled cost
- **Human-in-the-loop:** Flag low-confidence or high-risk outputs for human review before delivery

CI/CD Pipelines for AI



Continuous Integration

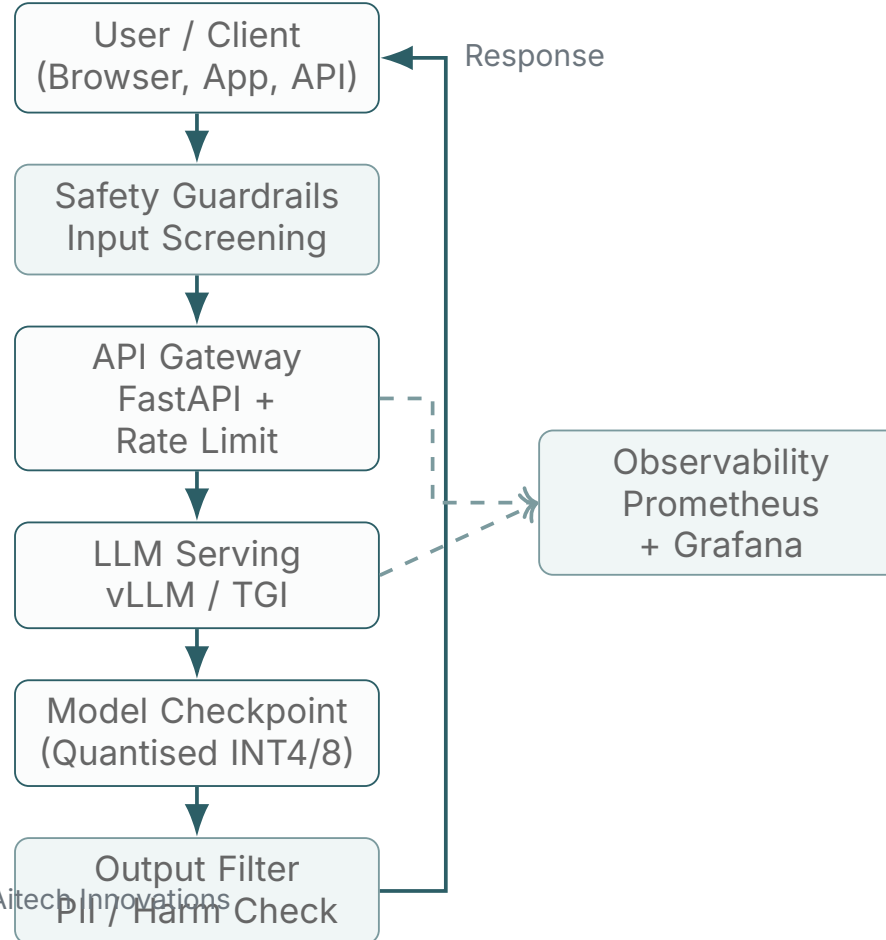
- Run automated eval suite on every prompt or model change
- Gate deployment on passing quality thresholds
- Track prompt version in Git alongside code
- Detect regressions before users do
- Tools: GitHub Actions, GitLab CI, Weights & Biases

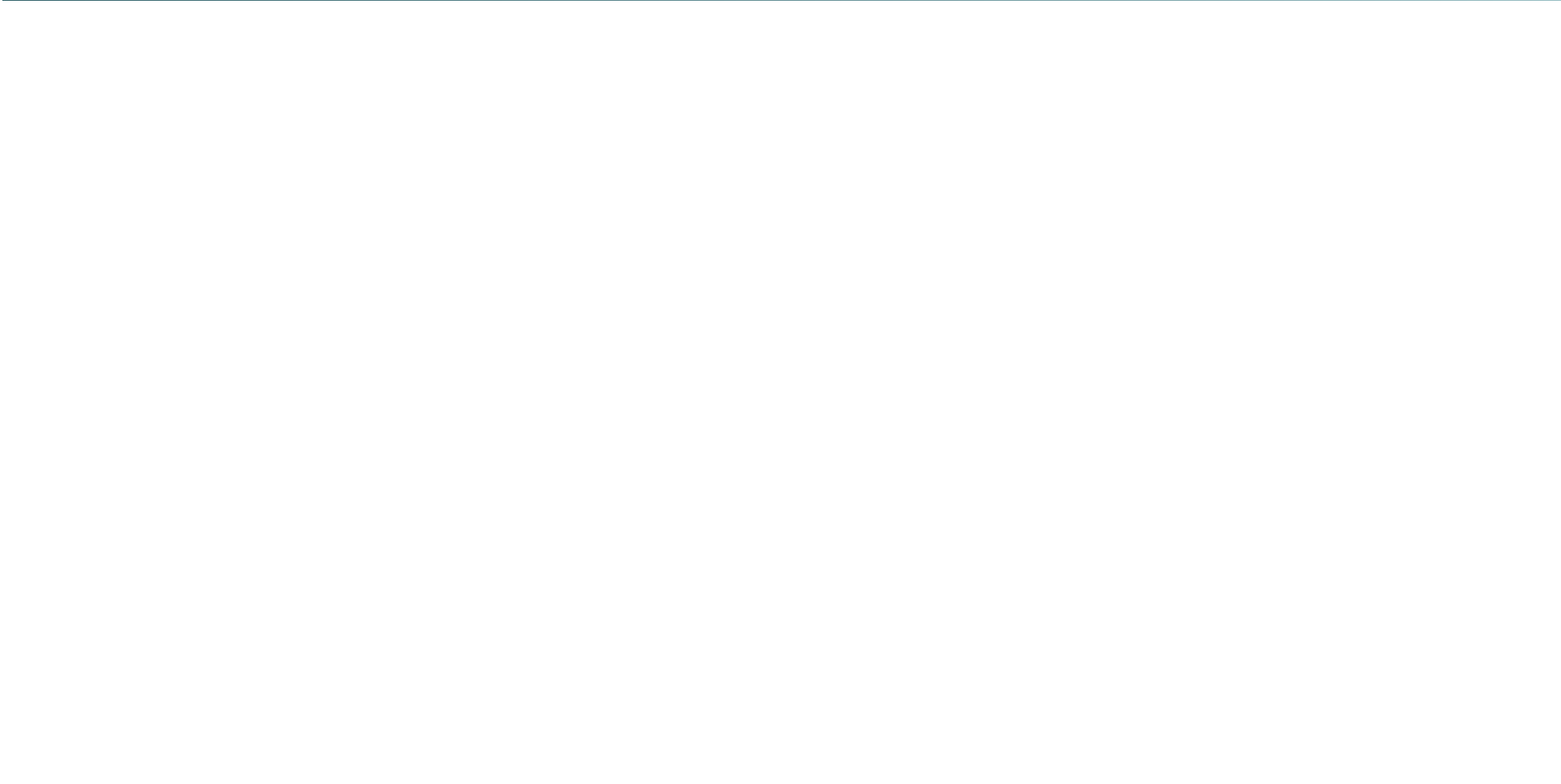
Continuous Deployment

- **Blue-green deployment:** Route traffic gradually from old to new model version
- **Canary release:** Send 5% of traffic to new model; monitor quality before full rollout
- **Model registry:** Version-controlled checkpoints with metadata (eval scores, training data hash)
- **Rollback:** Instant revert to previous version if quality degrades
- Tools: MLflow, ClearML, Seldon, BentoML



Production AI Stack: End-to-End Architecture







Application: AI Embedded in Developer Tools

GitHub Copilot (Level 4)

- Integrated directly into VS Code, JetBrains, Neovim
- LLM API call fires within 200ms of keystroke pause
- Prompt assembled from file context, cursor position, open tabs
- Telemetry tracks acceptance rate, latency, and token costs at scale
- Infrastructure: Azure OpenAI with private VNet, SLA of 99.9%

Integration Engineering Required

- IDE extension plugin lifecycle management
- Context window budget allocation (file vs. repo vs. diff)
- Streaming token delivery for real-time display
- Fallback to local model when network unavailable
- Cost monitoring: millions of API calls per day per enterprise



Application: Enterprise AI Search at Level 4

Beyond the RAG Prototype

A RAG prototype retrieves documents and generates answers. A production enterprise search system must additionally handle **access control** (a user in finance should not retrieve HR salary documents), **audit logging** (who asked what and what was returned, for compliance), **multi-tenant isolation** (different organisations on shared infrastructure), and **SLA enforcement** (p99 latency under 3 seconds even during peak load). These are purely Level 4 concerns.

Production Stack for Enterprise Search

- **Vector DB:** Pinecone or Azure AI Search with namespace-based tenant isolation
- **Access control:** Row-level security filters applied at retrieval time
- **Audit log:** Every query, retrieved chunk, and response stored with user identity
- **Observability:** RAGAS metrics (faithfulness, answer relevance) tracked per tenant



Application: Consumer Conversational AI at Scale

ChatGPT / Claude – Production Scale

- 100M+ daily active users (ChatGPT as of 2024)
- Infrastructure: multi-region Azure clusters with custom GPUs (Microsoft/OpenAI)
- Streaming delivery: first token in <300ms via server-sent events
- Cost management: usage tiers, rate limits, abuse detection at query level
- Model versioning: A/B testing new model checkpoints live in production

Key Level 4 Engineering Choices

- **Context window management:** Truncate, summarise, or cache conversation history
- **Speculative decoding:** Draft small model generates tokens; large model verifies in parallel
- **Prompt caching:** System prompt cached – users pay only for new input tokens
- **Safety:** Real-time content classifier on every output stream; block before completion
- **Fallback:** Route to smaller model when GPU capacity is saturated



Application: AI Embedded in Business Workflows

The Invisible Integration

The most successful Level 4 deployments make AI **invisible** to end users. Google Search AI Overviews, Notion AI, Slack AI channel summaries, and Salesforce Einstein are all AI systems embedded so seamlessly into existing workflows that users simply experience better products – without thinking about foundation models, prompts, or RAG pipelines. This is the hallmark of Level 4 integration done right.

Deployed Level 4 Production Examples

- **Google Search AI Overviews:** LLM-generated summaries on every query; billions of requests/day
- **Notion AI:** In-document writing assistance; context assembled from page hierarchy
- **Slack AI:** Channel summarisation and search; tenant-isolated, GDPR-compliant
- **Klarna AI Agent:** Handles 67% of customer service chats; equivalent to 700 full-time agents



Knowledge Graph: Where This Concept Lives

Prerequisite Concepts

- Level 1: Foundation Models (Module 2)
- Level 2: Model Adaptation (Module 2)
- Level 3: Applications and Agents (Module 2)
- Generative vs. Discriminative Models (Module 1)
- Five-Level GenAI Framework (Module 1)

Concepts This Unlocks

- LLMOps Fundamentals (Module 9)
- Docker and GPU Infrastructure (Module 9)
- Safety Guardrails and Evaluation (Module 9)
- LangGraph Production Agents (Module 9)
- Capstone: Full Production System (Module 10)



Common Misconceptions

Critical Corrections

- **"Deployment is just uploading the model"** – Production deployment requires containerisation, serving framework configuration, load balancing, monitoring, and safety layers. Uploading a model file is only the starting point
- **"Quantisation always hurts quality"** – INT4 quantisation with AWQ or GPTQ typically causes $<1\%$ quality degradation on standard benchmarks while reducing VRAM by $\sim 4\times$ – a strongly favourable trade-off for most applications
- **"Safety is the model's responsibility"** – Model alignment reduces harmful outputs but provides no formal guarantees. Production guardrails are mandatory independent safety layers, especially in regulated industries
- **"One serving setup fits all use cases"** – Latency-critical chat needs low p99 latency; batch document processing needs high throughput. These require different server configurations, batching strategies, and hardware choices
- **"Monitoring is optional until something breaks"** – Silent quality degradation (prompt drift, hallucination rate increase) can persist for weeks without infrastructure alarms triggering; output-quality monitoring must be proactive

Session Summary



Key Takeaways

- Level 4 Integration adds reliability, throughput, cost control, and observability on top of Level 3 applications
- The LLMOps lifecycle – build, evaluate, deploy, monitor, iterate – must be automated and continuous
- Serving frameworks (vLLM), containerisation (Docker), and quantisation are the three pillars of efficient deployment
- Safety guardrails and cost controls operate independently of the model itself

What Comes Next

- **In Module 2:** Level 5 – Innovation (the frontier of GenAI)
- **Module 9:** Full LLMOps deep-dive – Docker labs, vLLM configuration, Prometheus dashboards
- **Module 10:** Deploy your Capstone as a complete production system with all Level 4 components
- **Explore now:** Run `ollama run llama3` locally – your first self-hosted LLM serving instance

Thank You

Module 2: Generative AI – Core Concepts & Architecture
Level 4: Integration · Prof. Sashikumaar Ganesan · IISc Bengaluru